楼层结构标注器 — 完整实施计划

> 版本：v1.0
>
> 日期：2026-05-01
>
> 关联规范：
>
> - `docs/frontend/FLOOR_STRUCTURE_ANNOTATOR_SPEC.md`
> - `docs/backend/FLOOR_MAP_API_SPEC.md`
> - `docs/backend/FLOOR_MAP_HYBRID_RENDERING_PLAN.md`
> - `docs/backend/FLOOR_MAP_EXTRACTOR_SPEC.md`
> - `docs/backend/schemas/floor_map.v2.schema.json`

---

1. 目标

为运营/资产管理员提供单楼层结构语义审核界面，在 DXF 自动抽取候选结构（电梯/楼梯/卫生间/核心筒等）的基础上，允许人工确认/修正/增删/拖动，审核通过后写回后端，并决定该楼层使用 `vector` 还是 `semantic` 渲染模式。

---

2. 前置条件评估

2.1 已就绪 ✅

条件	说明
Admin 框架	Vue 3 + TypeScript + Element Plus + Pinia + Axios client（含 ApiError、refresh subscriber queue）完整
API 契约文档	`FLOOR_MAP_API_SPEC.md` + `floor_map.v2.schema.json` 均已定稿
现有楼层视图	`BuildingDetailView.vue` 、 `FloorPlanView.vue` 可参照结构
视觉参照	`frontend/src/app/pages/FloorPlan.tsx` （~900 行 React 手绘风原型）
Phase 1 DXF 流水线	`split_dxf_by_floor.py` + `annotate_hotzone.py` 已完成，A 座 24 层 SVG + 热区 JSON 已生成
后端错误处理框架	`AppException` 继承体系 + `error_handler.dart` 全局中间件已完备
后端现有楼层 API	7 个端点（列表/创建/详情/CAD上传/热区/单元列表/图纸版本）已实现

2.2 尚未就绪 ❌（需本计划覆盖）

条件	影响
后端 3 个新 API 端点未实现	阻塞前端 Phase 3+ 真实联调
`floor_maps` 表不存在，`floors` 表缺 3 列	阻塞后端所有新功能
7 个 `FLOOR_MAP_*` 错误码未注册	阻塞前端错误码映射
`scripts/floor_map/` 目录不存在（5 个 detector）	无候选数据，标注工具左侧面板为空
`split_dxf_by_floor.py` 未集成 detector	无法生成 `candidates.json`
`router/guards.ts` 不存在	已决策：dirty 守卫 inline 写在 AnnotatorView，无需新建

2.3 命名决策

规范文档第 1 节与第 2 节命名不一致，统一采用**第 2 节**（详细节）命名：

文件	采用名称
主视图	`AnnotatorView.vue`
Pinia Store	`useFloorStructuresStore` / `floorStructuresStore.ts`

---

3. 依赖关系总览

```
0-A 数据库迁移 ──────────────────────────────────────────┐
0-B 错误码注册 （与 0-A 并行）                            │
                                                          ↓
                                                 0-C 后端模型扩展
                                                          ↓
                                                 0-D 后端 Repository
                                                          ↓
                                                 0-E 后端 Service
                                                          ↓
                                                 0-F 后端 Controller ──→ 前端 Phase 3 真实联调
0-A ──→ 0-G Python detector（可与 0-C 并行）──────────────┘

前端 Phase 1（类型/常量/API 模块）— 无依赖，立即开始
          ↓
前端 Phase 2（Pinia Store）
          ↓
前端 Phase 3（UI 组件，0-F 完成前用 mock fixture）
          ↓
前端 Phase 4（路由 + 入口按钮）
```

```
          ↓
前端 Phase 5（单元测试 + E2E）
```

---

4. 实施阶段

Phase 0 — 后端前置条件（最优先，阻塞前端 Phase 3+）

0-A 数据库迁移（无依赖，立即开始）

文件：`backend/migrations/NNN_create_floor_maps.sql`

新建 `floor_maps` 表：

```
CREATE TABLE floor_maps (
  floor_id                UUID PRIMARY KEY REFERENCES floors(id) ON DELETE CASCADE,
  schema_version          VARCHAR(8)   NOT NULL DEFAULT '2.0',
  viewport                JSONB,
  outline                 JSONB,
  structures              JSONB        NOT NULL DEFAULT '[]',
  windows                 JSONB        NOT NULL DEFAULT '[]',
  north                   JSONB,
  candidates              JSONB,
  candidates_extracted_at TIMESTAMPTZ,
  updated_at              TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_by              UUID         REFERENCES users(id)
);
```

`floors` 表新增 3 列：

```
ALTER TABLE floors
  ADD COLUMN render_mode             VARCHAR(16)  NOT NULL DEFAULT 'vector',
  ADD COLUMN floor_map_schema_version VARCHAR(8),
  ADD COLUMN floor_map_updated_at    TIMESTAMPTZ;
```

约束要求：

- `render_mode` CHECK IN (`'vector'` , `'semantic'`)
- 所有时间戳字段使用 `TIMESTAMPTZ` （UTC 存储）

0-B 错误码注册（无依赖，与 *0-A* 并行）

文件：`docs/backend/ERROR_CODE_REGISTRY.md`

新增 7 个错误码：

错误码	HTTP 状态码	触发场景
`FLOOR_MAP_CANDIDATES_NOT_GENERATED`	404	GET candidates 时 floor_maps.candidates 为空
`FLOOR_MAP_SCHEMA_UNSUPPORTED`	400	PUT structures 时 schema_version 不是 "2.0"

`FLOOR_MAP_COORDINATE_OUT_OF_RANGE`	400	结构 rect 坐标超出 viewport
`FLOOR_MAP_STRUCTURE_LIMIT_EXCEEDED`	400	structures 数量 > 200 或 windows > 100
`FLOOR_MAP_INVALID_STRUCTURE_TYPE`	400	type 不在 6 种枚举内
`FLOOR_MAP_NOT_READY_FOR_SEMANTIC`	422	PATCH render-mode=semantic 时 structures 为空
`INVALID_RENDER_MODE`	400	PATCH render-mode 的 mode 值非法

0-C 后端 Dart 模型扩展（依赖 0-A）

文件 1： `backend/lib/modules/assets/models/floor.dart`

`Floor` 类新增字段：

```
final String renderMode;              // 'vector' | 'semantic'
final String? floorMapSchemaVersion;  // '2.0'
final DateTime? floorMapUpdatedAt;
```

`Floor.fromMap()` 从 SQL 结果解析新字段； `toJson()` 在 GET /api/floors/:id 响应中包含。

文件 2（新建）： `backend/lib/modules/assets/models/floor_map.dart`

```
class FloorMap {
  final String floorId;
  final String schemaVersion;
  final Map<String, dynamic>? viewport;
  final Map<String, dynamic>? outline;
  final List<Map<String, dynamic>> structures;
  final List<Map<String, dynamic>> windows;
  final Map<String, dynamic>? north;
  final Map<String, dynamic>? candidates;
  final DateTime? candidatesExtractedAt;
  final DateTime updatedAt;
  final String? updatedBy;
}
```

0-D 后端 Repository（依赖 0-C）

文件 1： `backend/lib/modules/assets/repositories/floor_repository.dart`

- `findById()` — 返回结果中补充 `render_mode` 、 `floor_map_schema_version` 、 `floor_map_updated_at` 三列
- 新增方法 `updateRenderMode(String floorId, String mode)` — 参数化 UPDATE，同步更新 `floors.render_mode` + `floors.floor_map_updated_at`

文件 2（新建）： `backend/lib/modules/assets/repositories/floor_map_repository.dart`

```
class FloorMapRepository {
  // 读取已确认结构（PUT 保存后的数据）
  Future<FloorMap?> findByFloorId(String floorId);
```

```
  // 读取候选项（Python 抽取写入的 candidates 列）
  Future<Map<String, dynamic>?> findCandidates(String floorId);

  // 覆盖写入 structures/outline/windows/north
  Future<FloorMap> upsert(FloorMap map, String updatedBy);
}
```

所有 SQL 使用 `Sql.named()` 参数化查询，禁止字符串拼接。

0-E 后端 Service（依赖 0-D）

文件： `backend/lib/modules/assets/services/floor_service.dart`

新增 3 个方法：

`getCandidates(String floorId)`

```
1. 校验 floorId 格式（UUID）
2. 调 FloorMapRepository.findCandidates(floorId)
3. candidates 为 null → 抛 NotFoundException('FLOOR_MAP_CANDIDATES_NOT_GENERATED')
4. 返回 candidates JSON
```

`saveStructures(String floorId, Map payload, String updatedBy)`

```
前置校验（失败抛 ValidationException）：
  - outline 必填
  - structures 数量 ≤ 200
  - windows 数量 ≤ 100
  - 每个 elevator 必须有 code
  - 每个 restroom 必须有 gender
  - 所有 rect 坐标须在 viewport 内

通过后：
  FloorMapRepository.upsert(...)
  floors.floor_map_updated_at = NOW()
  floors.floor_map_schema_version = '2.0'
 写审计日志：action='floor_map.structures.update', entity_id=floorId
```

`switchRenderMode(String floorId, String mode, String userId)`

```
校验 mode IN ('vector', 'semantic')
若 mode == 'semantic'：
 检查 FloorMapRepository.findByFloorId(floorId).structures 非空
 否则抛 InvalidStateTransitionException('FLOOR_MAP_NOT_READY_FOR_SEMANTIC')
调 FloorRepository.updateRenderMode(floorId, mode)
写审计日志：action='floor_map.render_mode.change', entity_id=floorId
```

0-F 后端 Controller（依赖 0-E）

文件： `backend/lib/modules/assets/controllers/floor_controller.dart`

新增 3 个 handler，全部通过 RBAC 中间件，禁止在 handler 内直接返回 `Response`：

路由	权限	调用	成功响应
`GET /api/floors/:floorId/structures/candidates`	`assets.read`	`getCandidates`	`200 { data: FloorMapV2 }`
`PUT /api/floors/:floorId/structures`	`assets.write`	`saveStructures`	`204 No Content`
`PATCH /api/floors/:floorId/render-mode`	`assets.write`	`switchRenderMode`	`200 { data: { render_mode } }`

扩展 `GET /api/floors/:floorId` — 响应 `data` 中新增：

```
{
  "render_mode": "vector",
  "floor_map_schema_version": "2.0",
  "floor_map_updated_at": "2026-05-01T08:00:00Z"
}
```

0-G Python 结构抽取脚本（依赖 0-A，可与 0-C 并行）

目录：`scripts/floor_map/`（新建）

共用约定：

- 所有 detector 为纯函数，无副作用
- 失败返回 `[]` / `None`，不抛异常（调用层捕获日志）
- 统一函数签名 `(msp, region, scale_x, scale_y)`（`region` 来自 `split_dxf_by_floor.py` 已有的图框识别结果）
- 单层全部 detector 合计处理时间 ≤ 3 秒

`scripts/floor_map/__init__.py`（package 声明，空文件）

`scripts/floor_map/coordinate_mapper.py`（供其余 4 个 detector 共用）

```
def dxf_to_svg(dxf_x, dxf_y, region, scale_x, scale_y) -> tuple[float, float]:
    """DXF Model Space 坐标→ SVG viewport 坐标（含 Y 轴翻转）"""
    svg_x = (dxf_x - region.min_x) * scale_x
    svg_y = (region.max_y - dxf_y) * scale_y  # Y 翻转，与 annotate_hotzone.py 一致
    return svg_x, svg_y

def bbox_to_rect(min_x, min_y, max_x, max_y, region, scale_x, scale_y) -> dict:
    """DXF 包围盒→ SVG 坐标系下 {x, y, w, h}"""
    x, y2 = dxf_to_svg(min_x, max_y, region, scale_x, scale_y)  # 左上角
    x2, y = dxf_to_svg(max_x, min_y, region, scale_x, scale_y)  # 右下角
    return {"x": round(x, 2), "y": round(y, 2),
            "w": round(x2 - x, 2), "h": round(y2 - y, 2)}
```

`scripts/floor_map/outline_extractor.py`

```
def extract_outline(msp, region, scale_x, scale_y) -> dict | None:
```

检测策略：

1. 若 `region` 来自矩形图框（TK 图层 LWPOLYLINE，已由 `split_dxf_by_floor.py` 识别）→ 直接转换为 `{type: "rect", x, y, w, h}`
2. 若 `region` 为不规则区域（屋顶层等）→ 取图框多边形顶点，逐点映射 → `{type: "polygon", points: [[x,y],...]}`
3. 失败 → 返回 `None`，调用层记录 WARNING，不中断整体流程

`scripts/floor_map/structure_detector.py`

```
def detect_structures(msp, region, scale_x, scale_y) -> list[dict]:
```

逐类检测后合并，每项含 `type / rect / label / code? / gender? / confidence / source="auto"`：

类型	图层关键字	几何特征	补充规则
`core`	`WALL` / `CURTWALL` / 外墙	封闭 LWPOLYLINE，面积 >50 ㎡，位于平面中央 1/3 区域	相邻区域合并为单个大矩形
`elevator`	电梯 / `LIFT` / `ELEV`	INSERT 块或封闭矩形，宽高比 0.8~1.5，面积 4~25 ㎡	自动生成 `code`（E1/E2...，按 X 坐标排序）
`stair`	楼梯 / `STAIR`	封闭区域含折返线特征，面积 15~80 ㎡	—
`restroom`	卫 / `WC` / `TOILET` / `SANITARY`	封闭区域或 INSERT 块	无法推断性别时 `gender=null`
`equipment`	设备 / 机房 / `MECH` / `EQUIP`	封闭区域	—
`corridor`	走廊 / `CORRIDOR` / `PUBLIC`	封闭区域或轮廓内非房间覆盖推断	降级方案 confidence 较低

`confidence` 评分规则：图层名精确命中 → 0.85~1.0；关键字部分匹配 → 0.5~0.84；面积合理性加分 ±0.1。

`scripts/floor_map/column_detector.py`

```
def detect_columns(msp, region, scale_x, scale_y) -> list[dict]:
```

- 扫描图层含 柱 / `COLUMN` / `COL` 的封闭矩形或实心填充（SOLID/HATCH）
- **网格化去重**：相邻中心距 ≤ 500mm 的候选合并为一个，防止双线图层重复检测
- 上限 100 个/层，超出时按面积降序截断
- 输出：`{type: "column", rect: {x,y,w,h}, source: "auto", confidence}`

`scripts/floor_map/window_detector.py`

```
def detect_windows(msp, region, scale_x, scale_y) -> list[dict]:
```

- 扫描图层含 `窗` / `WINDOW` / `WIN` / `GLAZ` 的 LINE/LWPOLYLINE
- 过滤条件：线段中点到外轮廓边界距离 ≤ 200mm（DXF 单位）
- 按外轮廓中心原点判断方位，归入 `N / S / E / W` 四组
- 每条窗线输出 `{side, x, width, source: "auto"}`（x/width 为沿该边的局部坐标，单位 SVG px）
- 上限 100 个/层

扩展 `scripts/split_dxf_by_floor.py`

在现有 Stage 6（JSON 骨架输出）之后新增 **Stage 7**：候选结构抽取：

```
Stage 7: 候选结构抽取（仅 --extract-structures 开启时执行）
  1. 调用 coordinate_mapper 初始化 scale_x / scale_y
  2. outline   = outline_extractor.extract_outline(...)
  3. structures = structure_detector.detect_structures(...)
  4. columns    = column_detector.detect_columns(...)
  5. windows    = window_detector.detect_windows(...)
  6. 组装 candidates = {
       schema_version: "2.0",
       viewport: {...},
       outline,
       structures: structures + columns,
       windows
     }
  7. 写入 A座_F*.candidates.json（同目录）
  8. 若提供 --db-url → psql UPSERT INTO floor_maps(floor_id, candidates,
candidates_extracted_at)
```

新增 CLI 参数：

参数	默认值	说明
`--extract-structures`	off	开启 Stage 7，不影响现有流水线
`--db-url`	空	PostgreSQL 连接串，用于直写 floor_maps 表

单层总处理时限：原切分 + Stage 7 ≤ 10 秒。

单元测试： `scripts/floor_map/tests/`

测试文件	验证内容
`test_coordinate_mapper.py`	已知 DXF 坐标 → 期望 SVG 坐标；Y 翻转正确

`test_outline_extractor.py`	mock msp 含 TK 图层矩形 → 输出 `{type: "rect", ...}`
`test_structure_detector.py`	mock INSERT 块含电梯图层 → 检测出 `elevator` ；code 自动生成
`test_column_detector.py`	12 个柱位含 2 个相邻重复（距离 <500mm）→ 输出 11 个
`test_window_detector.py`	北侧 3 扇窗 → `side=N` ，x/width 正确

运行方式： `python -m pytest scripts/floor_map/tests/ -v`

Phase 1 — 前端：类型 / 常量 / API 模块（无依赖，立即开始）

1-1 TypeScript 类型定义

文件（新建）： `admin/src/types/floorMap.ts`

```
export type StructureType =
  | 'core' | 'elevator' | 'stair' | 'restroom'
  | 'shaft' | 'column' | 'corridor' | 'lobby';

export interface Rect { x: number; y: number; w: number; h: number }

export interface Structure {
  type: StructureType;
  rect: Rect;
  label?: string;
  code?: string;          // elevator 必填
  gender?: 'M' | 'F' | 'U';  // restroom 必填
  source: 'auto' | 'manual';
  confidence?: number;    // 仅 auto 时存在
}

export interface WindowSegment {
  side: 'N' | 'S' | 'E' | 'W';
  x: number;
  width: number;
  source?: 'auto' | 'manual';
}

export interface Outline {
  type: 'rect' | 'polygon';
  x?: number; y?: number; w?: number; h?: number;  // type=rect
  points?: [number, number][];                      // type=polygon
}

export interface FloorMapV2 {
  schema_version: '2.0';
  viewport?: { width: number; height: number };
  outline: Outline;
  structures: Structure[];
  windows?: WindowSegment[];
```

```
  north?: { x: number; y: number; rotation_deg?: number };
}
```

字段定义直接对应 `floor_map.v2.schema.json`，修改 schema 后必须同步更新此文件。

1-2 API 路径常量

文件：`admin/src/constants/api_paths.ts`（增补）

```
// 楼层结构标注（Floor Map v2）
floorStructureCandidates: (floorId: string) =>
  `/api/floors/${floorId}/structures/candidates`,
floorStructures: (floorId: string) =>
  `/api/floors/${floorId}/structures`,
floorRenderMode: (floorId: string) =>
  `/api/floors/${floorId}/render-mode`,
```

组件/store 内禁止硬编码路径字符串，统一从此处引用。

1-3 UI 常量

文件：`admin/src/constants/ui_constants.ts`（增补）

```
import type { StructureType } from '@/types/floorMap';

// 结构类型颜色（全部使用 CSS 变量，禁止 #xxx 硬编码）
export const STRUCTURE_TYPE_COLORS: Record<StructureType, string> = {
  core:      'var(--floor-core)',
  elevator:  'var(--floor-elevator)',
  stair:     'var(--floor-stair)',
  restroom:  'var(--floor-restroom)',
  column:    'var(--floor-column)',
  corridor:  'var(--floor-corridor)',
  shaft:     'var(--floor-shaft)',
  lobby:     'var(--floor-lobby)',
};
```

CSS 变量在 `admin/src/styles/tokens.scss`（或全局样式）中定义默认值：

```
:root {
  --floor-core:      rgba(150, 150, 150, 0.4);
  --floor-elevator:  rgba(100, 150, 255, 0.4);
  --floor-stair:     rgba(100, 200, 150, 0.4);
  --floor-restroom:  rgba(200, 150, 200, 0.4);
  --floor-column:    rgba(80, 80, 80, 0.6);
  --floor-corridor:  rgba(200, 200, 160, 0.3);
  --floor-shaft:     rgba(160, 130, 100, 0.4);
  --floor-lobby:     rgba(130, 190, 230, 0.3);
}
```

1-4 API 函数模块

文件（新建）：`admin/src/api/modules/floorStructures.ts`

```
import { apiGet, apiPut, apiPatch } from '@/api/client';
import { API_PATHS } from '@/constants/api_paths';
import type { FloorMapV2 } from '@/types/floorMap';

// 获取候选项（DXF 自动抽取）
export const getCandidates = (floorId: string) =>
  apiGet<FloorMapV2>(API_PATHS.floorStructureCandidates(floorId));

// 获取已确认结构（上次 PUT 保存的数据）
export const getConfirmedStructures = (floorId: string) =>
  apiGet<FloorMapV2>(API_PATHS.floorStructures(floorId));

// 保存审核后的结构（覆盖写）
export const putStructures = (floorId: string, payload: FloorMapV2) =>
  apiPut<void>(API_PATHS.floorStructures(floorId), payload);

// 切换渲染模式
export const patchRenderMode = (floorId: string, mode: 'vector' | 'semantic') =>
  apiPatch<{ render_mode: string }>(API_PATHS.floorRenderMode(floorId), { mode });
```

Phase 2 — 前端：Pinia Store（依赖 Phase 1）

文件（新建）：`admin/src/stores/floorStructuresStore.ts`

Setup 风格，≤ 200 行。Store state 固定字段：

```
const candidates   = ref<FloorMapV2 | null>(null)   // GET candidates 返回值（原始，只读）
const draft        = ref<FloorMapV2 | null>(null)   // 当前编辑草稿（深拷贝）
const renderMode   = ref<'vector' | 'semantic'>('vector')
const loading      = ref(false)
const saving       = ref(false)
const error        = ref<string | null>(null)
const dirty        = ref(false)
const selectedIndex = ref<number | null>(null)      // 当前选中结构的下标

// 历史栈（支持撤销/重做）
const history      = ref<FloorMapV2[]>([])
const historyIndex = ref(-1)                        // 上限 50 条
```

Actions（全部用 `try/catch` 包装，错误处理统一模式）：

```
catch (e) {
  error.value = e instanceof ApiError ? e.message : '操作失败，请重试';
}
```

Action	说明
`loadCandidates(floorId)`	GET candidates；失败降级 GET confirmed；仍失败显示空状态
`loadConfirmed(floorId)`	GET 已确认数据

`save(floorId)`	前端 validate → PUT structures → `dirty=false`；412 时提示刷新
`setRenderMode(floorId, mode)`	PATCH render-mode；`dirty=true` 时禁用（由组件层判断）
`addStructure(s)`	push 到 `draft.structures`，记录历史，`dirty=true`
`updateStructure(idx, patch)`	更新指定下标，记录历史，`dirty=true`
`removeStructure(idx)`	删除，记录历史，`dirty=true`
`undo()`	`historyIndex--`，恢复 draft
`redo()`	`historyIndex++`，前进 draft
`reset()`	恢复到 candidates 初始状态，清空历史，`dirty=false`

内联 `validate(map: FloorMapV2): string | null`（与后端 §0-E 校验保持一致）：

```
if (!map.outline) return 'outline 缺失';
if (map.structures.length > 200) return '结构数量超过 200';
for (const s of map.structures) {
  if (s.rect.w <= 0 || s.rect.h <= 0) return '矩形尺寸非法';
  if (s.type === 'elevator' && !s.code) return '电梯必须填写编号';
  if (s.type === 'restroom' && !s.gender) return '卫生间必须选择性别';
}
return null;
```

文件：`admin/src/stores/index.ts` — 导出 `useFloorStructuresStore`。

Phase 3 — 前端：UI 组件（依赖 Phase 2，后端 0-F 完成前用 mock fixture）

目录结构：

```
admin/src/views/assets/floor-structures/
  AnnotatorView.vue                  # 容器页（≤ 250 行）
  components/
    Toolbar.vue                      # 顶部工具栏
    CandidatesPanel.vue              # 左侧候选清单（240px）
    CanvasStage.vue                  # 中央 SVG 画布
    InspectorPanel.vue               # 右侧属性面板（320px）
    RenderModeSwitch.vue             # 渲染模式切换
  composables/
    useCanvasInteraction.ts          # 拖拽/缩放/键盘逻辑
```

3-1 `AnnotatorView.vue`（容器，≤ 250 行）

- `onMounted`：解析路由参数 `floorId`，调 `store.loadCandidates(floorId)`
- 三栏布局：`CandidatesPanel`（左 240px）/ `CanvasStage`（居中，flex-1）/ `InspectorPanel`（右 320px），顶部 `Toolbar`

- `beforeRouteLeave` inline dirty 守卫：

```
onBeforeRouteLeave(() => {
  if (!store.dirty) return true;
  return ElMessageBox.confirm('有未保存的修改，确认离开？', '提示', {
    confirmButtonText: '离开', cancelButtonText: '留下',
    type: 'warning'
  }).then(() => true).catch(() => false);
});
```

3-2 `Toolbar.vue`

按钮及快捷键：

按钮	Action	快捷键
新增矩形	设置 canvas 为 `draw` 模式	`N`
删除	`store.removeStructure(selectedIndex)`	`Delete`
撤销	`store.undo()`	`Cmd/Ctrl + Z`
重做	`store.redo()`	`Shift + Cmd/Ctrl + Z`
保存	`store.save(floorId)` ；禁用条件：validate 不通过	`Cmd/Ctrl + S`
`RenderModeSwitch`	嵌入工具栏右侧	—

3-3 `CandidatesPanel.vue`（左侧，240px）

- 展示 `store.candidates.structures` （仅 `source=auto` 的候选项）
- 每项显示：类型图标 / label / `confidence` 百分比 Tag（低于 0.5 显示 warning 色）
- 点击 → `store.addStructure(s)` 将候选加入画布 draft
- 已加入的候选项显示灰色「已添加」状态，不可重复添加
- 空状态文案：「该楼层尚未生成候选项，请先上传 DXF 并运行抽取流程」

3-4 `composables/useCanvasInteraction.ts`

抽离所有 pointer/keyboard 事件逻辑，不直接操作 DOM，通过返回的 reactive 状态供 `CanvasStage` 使用：

```
export function useCanvasInteraction(store: ReturnType<typeof useFloorStructuresStore>) {
  const mode = ref<'select' | 'draw' | 'pan'>('select');
  const transform = reactive({ scale: 1, translateX: 0, translateY: 0 });
  const drawPreview = ref<Rect | null>(null);  // 框选新建时的预览矩形

  // 缩放：0.5x ~ 3x，滚轮 / 触控板捏合
  // 平移：按住 Space + 拖拽空白区域
  // 拖动结构：选中后按下中心区域
  // 键盘移动：方向键 1px，Shift+方向键 10px
  // 所有 drag 操作使用 requestAnimationFrame 节流，mouseup 时一次性 commit store
}
```

3-5 `CanvasStage.vue`（SVG 画布）

```
<div class="canvas-container" @wheel="onWheel" @pointerdown="onPointerDown" ...>
  <svg :viewBox="`0 0 ${viewport.width} ${viewport.height}`"
       :style="{ transform: `scale(${scale}) translate(${tx}px, ${ty}px)` }">

    <!-- Layer 1: outline 边框 -->
    <rect v-if="outline.type==='rect'" ... fill="var(--floor-outline-bg)" />
    <polygon v-else :points="outlinePoints" ... />

    <!-- Layer 2: structures 结构矩形（按类型上色，fill-opacity: 0.3） -->
    <g v-for="(s, i) in draft.structures" :key="i" @click="selectStructure(i)">
      <rect :x="s.rect.x" :y="s.rect.y" :width="s.rect.w" :height="s.rect.h"
            :fill="STRUCTURE_TYPE_COLORS[s.type]"
            :stroke="s.source==='manual' ? currentColor : 'none'"
            :stroke-width="s.source==='manual' ? 2 : 0" />
    </g>

    <!-- Layer 3: windows 窗洞（橙色线，4px） -->
    <line v-for="w in draft.windows" ... stroke="var(--floor-window)" stroke-width="4" />

    <!-- Layer 4: 选中高亮框（虚线） -->
    <rect v-if="selectedIndex !== null" ... stroke-dasharray="4 4" />

    <!-- Layer 5: 拖动控制点（8 个，仅选中时显示） -->
    <circle v-for="handle in handles" ... />

    <!-- Layer 6（draw 模式）: 框选预览矩形 -->
    <rect v-if="drawPreview" ... stroke-dasharray="6 3" fill="none" />
  </svg>
</div>
```

坐标系与后端 JSON 完全一致（已经过 Y 翻转，左上为原点），不在前端做坐标变换。

100×100 单位网格（默认开启，可通过工具栏关闭）。

3-6 `InspectorPanel.vue`（右侧，320px）

根据 `selectedStructure.type` 动态渲染字段：

字段	组件	校验
`type`	`el-select`（6 种枚举）	必填
`label`	`el-input`	选填，maxLength 32
`code`	`el-input`	`elevator` 时必填，正则 `^[A-Z]\d{1,3}$`
`gender`	`el-radio-group`（男/女/未知）	`restroom` 时必填
`rect.x/y/w/h`	`el-input-number`	w/h ≥ 1，不超过 viewport
`confidence`	只读 `el-tag`（百分比）	仅 `auto` 显示

校验失败时字段下方显示错误文字，同时禁用 Toolbar 保存按钮（通过 store 内 validate 函数驱动）。

3-7 `RenderModeSwitch.vue`

```
<el-switch
  v-model="isSemantic"
  active-text="语义" inactive-text="矢量"
  :disabled="store.dirty || store.saving"
  @change="onModeChange"
/>
```

- `dirty=true` 时禁用并 tooltip 提示「请先保存当前修改」
- 切换成功后 `store.renderMode` 更新，父组件楼层卡片 `render_mode` Tag 随之刷新

Phase 4 — 前端：路由 + 入口按钮（依赖 Phase 3）

4-1 路由注册

文件：`admin/src/router/index.ts`（增补）

```
{
  path: '/assets/buildings/:buildingId/floors/:floorId/structures',
  name: 'FloorStructureAnnotator',
  component: () => import('@/views/assets/floor-structures/AnnotatorView.vue'),
  meta: { requiresAuth: true }
}
```

位置：插入在 `/assets/buildings/:id` 路由的子路由之后，保持路由树层级清晰。

4-2 楼层卡片入口

文件：`admin/src/views/assets/BuildingDetailView.vue`（修改）

楼层卡片右上角新增：

1. `render_mode` 状态 Tag：`vector` → `el-tag type="info"`；`semantic` → `el-tag type="success"`
2. 「结构标注」按钮（仅 `assets.write` 权限显示）：

```
<el-button
  v-if="authStore.hasPermission('assets.write')"
  size="small"
  @click="router.push({ name: 'FloorStructureAnnotator',
    params: { buildingId: floor.buildingId, floorId: floor.id } })"
>
 结构标注
</el-button>
```

Phase 5 — 测试（依赖 Phase 2-4）

5-1 Pinia Store 单元测试

文件（新建）：`admin/src/stores/__tests__/floorStructuresStore.spec.ts`

使用 `vitest` + `@pinia/testing`：

测试用例	验证点
`addStructure` 后 `dirty === true`	dirty 状态正确
`addStructure` → `undo` → `draft.structures` 恢复	撤销正确
`undo` → `redo` → `draft.structures` 前进	重做正确
`removeStructure` 删除指定下标	删除正确
`updateStructure` 更新 rect	更新正确
`save` 失败时 `error.value` 被赋值	错误处理
`save` 成功时 `dirty === false`	dirty 清除
`validate` 缺 outline → 返回错误字符串	前置校验
`validate` elevator 无 code → 返回错误字符串	前置校验
`reset` 后 `dirty === false`，`draft` 恢复初始	重置正确
历史栈深度 > 50 时最早记录被丢弃	内存保护

5-2 E2E 测试

文件（新建）：`admin/e2e/floor-structures.test.ts`

使用 Playwright，通过 fixtures 注入 mock 候选数据：

场景	步骤
加载候选项	进入页面 → 左侧候选清单非空 → 结构自动渲染在画布
添加候选 → 保存	点击候选项 → 修改 type/label → 点保存 → 刷新后结构保留
框选新建 restroom	工具栏「新增矩形」→ 画布拖框 → 设为 restroom + gender=F → 保存
渲染模式切换	保存后切换 vector→semantic → 楼层卡片 render_mode Tag 更新为「语义」
离开守卫	修改后点导航跳转 → 弹出「有未保存修改」弹窗 → 点取消 → 留在页面

5. 完整文件清单

后端（Phase 0）

文件	操作
`backend/migrations/NNN_create_floor_maps.sql`	新建
`docs/backend/ERROR_CODE_REGISTRY.md`	增补 7 个错误码
`backend/lib/modules/assets/models/floor.dart`	增 3 字段
`backend/lib/modules/assets/models/floor_map.dart`	新建
`backend/lib/modules/assets/repositories/floor_repository.dart`	增 `updateRenderMode` 方法
`backend/lib/modules/assets/repositories/floor_map_repository.dart`	新建
`backend/lib/modules/assets/services/floor_service.dart`	增 3 方法 + 审计日志
`backend/lib/modules/assets/controllers/floor_controller.dart`	增 3 handler，扩展 GET

Python 抽取脚本（Phase 0-G）

文件	操作
`scripts/floor_map/__init__.py`	新建
`scripts/floor_map/coordinate_mapper.py`	新建
`scripts/floor_map/outline_extractor.py`	新建
`scripts/floor_map/structure_detector.py`	新建
`scripts/floor_map/column_detector.py`	新建
`scripts/floor_map/window_detector.py`	新建
`scripts/floor_map/tests/test_coordinate_mapper.py`	新建
`scripts/floor_map/tests/test_outline_extractor.py`	新建
`scripts/floor_map/tests/test_structure_detector.py`	新建
`scripts/floor_map/tests/test_column_detector.py`	新建
`scripts/floor_map/tests/test_window_detector.py`	新建
`scripts/split_dxf_by_floor.py`	增 Stage 7 + 2 个 CLI 参数

前端（Phase 1-5）

文件	操作
`admin/src/types/floorMap.ts`	新建
`admin/src/constants/api_paths.ts`	增补 3 条路径
`admin/src/constants/ui_constants.ts`	增补 `STRUCTURE_TYPE_COLORS`
`admin/src/api/modules/floorStructures.ts`	新建
`admin/src/stores/floorStructuresStore.ts`	新建
`admin/src/stores/index.ts`	导出新 store
`admin/src/views/assets/floor-structures/AnnotatorView.vue`	新建
`admin/src/views/assets/floor-structures/components/Toolbar.vue`	新建
`admin/src/views/assets/floor-structures/components/CandidatesPanel.vue`	新建
`admin/src/views/assets/floor-structures/components/CanvasStage.vue`	新建
`admin/src/views/assets/floor-structures/components/InspectorPanel.vue`	新建
`admin/src/views/assets/floor-structures/components/RenderModeSwitch.vue`	新建
`admin/src/views/assets/floor-structures/composables/useCanvasInteraction.ts`	新建
`admin/src/router/index.ts`	增 1 条路由
`admin/src/views/assets/BuildingDetailView.vue`	增入口按钮 + render_mode Tag
`admin/src/stores/__tests__/floorStructuresStore.spec.ts`	新建
`admin/e2e/floor-structures.test.ts`	新建

6. 验证方案

层次	命令	通过标准
Python 脚本单测	`python -m pytest scripts/floor_map/tests/ -v`	5 个测试全绿
后端单元测试	`cd backend && dart test test/modules/assets/`	新增 Service/Repository 测试全绿
前端 Store 单测	`pnpm --filter admin test:unit`	11 个 store 测试用例全绿
TypeScript 检查	`pnpm --filter admin lint`	`strict: true` 无报错
本地端到端联调	手动操作	候选加载 → 拖拽 → 保存 → `dirty=false` → 模式切换成功
Playwright E2E	`pnpm --filter admin test:e2e floor-structures`	5 个场景全通过

7. 技术风险与决策记录

风险	决策
`router/guards.ts` 不存在	`beforeRouteLeave` inline 写在 `AnnotatorView.vue`，不新建 guards.ts
后端 API 未就绪时前端无法真实联调	Phase 3 期间使用 `admin/e2e/fixtures/candidates.mock.json` mock 数据，Phase 0-F 完成后切换真实 API
Python detector 识别率难以保证	confidence 字段透传给前端显示，低置信度候选不自动加入画布，由人工决策；detector 失败返回空列表不中断流程
SVG 元素总数上限	structures ≤ 200 + windows ≤ 100 + 控制点 8 = 约 310，原生 SVG 足够，不引入虚拟化
历史栈内存占用	深度上限 50 条，每条为 FloorMapV2 完整深拷贝（预估 < 5MB，可接受）
坐标一致性	SVG viewBox 坐标系与后端 JSON 完全一致（Y 轴已翻转，左上为原点），前端不做二次坐标变换
颜色规范	禁止 `#xxx` 硬编码，全部通过 CSS 变量；`STRUCTURE_TYPE_COLORS` 只引用 `var(--floor-*)`